

Analysing a Progressive Ratio assay with pyflic

A guide for researchers running Progressive Ratio (PR) experiments on the FLIC. It explains what pyflic measures, which settings you control and how to choose them, how to read each figure, and what the statistics do and do not let you conclude.

You do not need to read code to use it. For installing and starting pyflic, see [Getting Started](#). The in-app help topic *Progressive Ratio experiments* ([pyflic help](#)) covers the same ground as a concise reference. The developer's view is in [progressive-ratio-implementation.md](#).

1. The assay in one page

Each DFM's six chambers form three **chamber groups**: chambers 1+2, 3+4 and 5+6. The two chambers of a group share one light circuit and one treatment.

- The **paired** fly controls the light. When it feeds at the **sucrose well** (always well A), the light comes on, typically driving optogenetic stimulation.
- The **yoked** fly, its partner, is lit at exactly the same moments, whatever it does itself. It receives the same light but cannot earn it.

The recording has two phases:

1. **Training.** Closed loop: every sucrose feed by the paired fly turns the light on. The firmware marks training in the data, and each group finishes training at its own time, when its paired fly has done so.
2. **Test.** The firmware demands progressively more licks for each light event. A fly that finds the light rewarding keeps working as the price rises, up to a point.

The analysis asks three questions:

Question	Main measure
Does the paired fly feed differently from a fly that got the same light for free?	the paired – yoked difference , within each group
How far did the paired fly keep working as the requirement rose?	the breaking point
How long did each fly keep feeding at the sucrose well?	sucrose persistence , for both flies

Two rules run through everything:

- **Differences are always taken within a chamber group, never between group means.** The yoked fly is the paired fly's own control: same device, same light, same moments.
- **Time runs from each group's training end**, not from the start of the recording.

2. Before you start

Data you need

- **Version-3 DFM files with light data (OptoColl1).** The training flag and the light state both live there. Without them no group ever finishes training, and every group is removed.
- **Both wells named:** well A is the sucrose well, well B the yeast well.

Setting up the experiment

For each DFM, the config states which chamber of each group is paired, and which side of the chamber well A (sucrose) is on:

```
dfms:
  1:
    params: {pi_direction: left}      # which side well A (sucrose) is on
    paired_chambers: [1, 4, 5]       # exactly one chamber from each group
    chambers: {1: Ctrl1, 2: Ctrl1, 3: Exp, 4: Exp, 5: Exp, 6: Exp}
```

- **paired_chambers** must name exactly one chamber from each of 1–2, 3–4 and 5–6. The yoked chamber is the other one and is never written.
- **Both chambers of a group must carry the same treatment.** pyflic refuses to load a config where they differ.
- **Paired and yoked are roles, not treatments.** Don't create a "paired" treatment or design factor; pyflic adds a **Role** column itself.
- In the **Config Editor**, the paired chambers are three pickers on each DFM tab.

Setting up the Project

Put the recordings that answer one question into one **Project**. Its **project.yaml** holds the shared **design**, which every member inherits:

```
# project.yaml
name: my_pr_study
design:
  global:
    experiment_type: ProgressiveRatio
    well_names: {A: Sucrose, B: Yeast}
    constants:
      pr_break_gap_min: 120
      # pr_test_window_min: 720      # optional; see section 4
    experimental_design_factors:
      Genotype: [Ctrl1, Exp]
```

Edit it in the Hub with **Project panel** → **Project design....** Its sections hold every setting in section 4. **Set the analysis constants here, once.** Every member is then analysed under the same rules, which pooling requires.

3. Running the analysis

For each member (experiment):

1. **Hub → Analyze → Basic analysis.** This runs everything below and writes the tables and figures into the member's `analysis/` folder.
2. **Hub → Analyze → PDF report**, or `pyflic report <member folder>`. This writes the experiment report.

For the Project:

3. **Project panel → Build combined analysis.** This stacks every member's tables and runs the pooled statistics (`<project>_Stats.txt`).
4. **Project panel → Create report**, or `pyflic report <project folder>`. This writes the Project Report.

Every step is also a Script Editor action, so a Project Script or `pyflic batch` can run it unattended. **Re-run basic analysis after changing any setting.** Members analysed before a setting changed keep their old numbers until you do.

What basic analysis does, in order

1. **Training.** It finds each group's training end (section 5.1).
2. **Quality control and exclusions.** It checks training and light for every group, and removes chambers that fail (sections 5.2 and 5.3).
3. **Feeding summaries.** It computes the standard two-well metrics (licks, events, durations, preference index) per chamber, for the whole recording and separately for the Training and Test phases.
4. **Paired – yoked difference.** One row per chamber group per phase (section 5.4).
5. **Cumulative difference curve** (section 5.5).
6. **Breaking point** and **sucrose persistence** (sections 5.6 and 5.7).
7. **Figures** and the text summary (`summary.txt`).

4. Settings you control

All of these are in the Project's design (**Project design...**), except the figure bin size, which is set per plot action. The defaults are chosen to be sensible for a typical overnight recording. Change one only for a reason you can state.

Study design

Setting	What it does	Choosing it
<code>paired_chambers</code> (per DFM)	which chamber of each group controls the light	from your rig setup; must match what the firmware did
<code>pi_direction</code> (per DFM)	which side well A, the sucrose well, is on	from your rig setup; may differ between DFMs
treatments (<code>chambers:</code>)	what each group received	the same for both chambers of a group
<code>experimental_design_factors</code>	factors such as genotype and drug, crossed	the first factor is the x axis of the dot plots

Setting	What it does	Choosing it
<code>well_names</code>	labels for wells A and B in plots and reports	well A must be the sucrose well
<code>transform_licks</code>	transforms lick counts in the summary tables	the cumulative curves always use raw counts regardless

Detection parameters

`feeding_threshold`, `feeding_minimum`, `tasting_minimum` and the other detection parameters decide which signal deflections count as licks and feeding events. They matter more in this assay than in most, because the **light QC** and the **breaking point** both depend on whether a light event was backed by at least one lick. See the *Parameter reference* help topic. Change them for the whole Project, never for one member.

Exclusion

Setting	Default	Effect
<code>min_untransformed_licks_cutoff</code>	20	removes a chamber whose sucrose or yeast well has fewer licks than this over the whole recording, counted before any transform
<code>max_med_duration_cutoff</code>	13.0	removes a chamber whose median feeding-event duration at either well is above this (a sign of a stuck or bleeding signal)
<code>max_events_cutoff</code>	150000	removes a chamber with an implausible number of events at either well
<code>require_training_complete</code>	true	removes both chambers of a group whose paired fly never finished training
<code>exclude_failed_pr_groups</code>	true	removes both chambers of a group that fails the light QC

Light QC thresholds

Setting	Default	Meaning
<code>pr_lick_free_run</code>	5	this many lick-free Test light events in a row fails the group as self-triggered
<code>pr_trend_min_events</code>	5	the fewest Test light events needed to judge the lick trend
<code>pr_trend_min_rho</code>	0.3	the licks-per-event trend must reach this rank correlation, or the group gets a warning
<code>pr_resting_level_rise</code>	15	a rise in the sucrose well's resting signal of this many counts is a warning

Setting	Default	Meaning
<code>pr_resting_level_ratio</code>	3	a sucrose well resting this many times higher than the DFM's other sucrose wells is a warning

Breaking point

Setting	Default	Meaning
<code>pr_break_gap_min</code>	120 minutes	the pause that counts as "the fly stopped". It is used for both the breaking point and sucrose persistence
<code>pr_test_window_min</code>	off	caps every group's Test window at the same length

Choosing `pr_break_gap_min`. The breaking point moves with this value, so pyflic always shows every group's breaking point at 60, 120 and 240 minutes beside your setting (in `summary.txt` and the *Breaking point* CSV log). Look at that table **before** you settle on a gap:

- If the ranking of treatments is the same at every gap, the result is robust.
- If it changes, report that.

Pick the gap on biological grounds, such as how long a fly normally goes between feeding bouts, and not by choosing whichever value gives the smallest p-value. pyflic does not know the time of day, so a long night pause ends a count like any other.

Choosing `pr_test_window_min`. Each group is watched from its own training end to the end of the recording, so a group that trained late gets less Test time and is more likely to be censored (section 5.6). The `TestMinutes` column of `pr_breaking_point.csv` shows each group's window. If those windows differ a lot, set `pr_test_window_min` to a value no longer than the shortest one: every group is then judged over the same span. It must be comfortably longer than `pr_break_gap_min`, or almost every group will be censored.

Figures

The cumulative-curve actions (*Cumulative difference curve*, *Training-aligned traces*) take a **bin size**, default 1 minute. It changes only the smoothness of the curve, not any statistic.

5. The analyses

5.1 Training

A group's **training end** is the last minute its paired fly's sucrose well carries the firmware's training flag. The other three wells of the group normally clear at the same moment. If they don't, the difference appears as a *training-flag note* in `summary.txt` and the report. These notes are QC information, not errors: the analysis always follows the paired sucrose well.

A group whose paired fly **never finished training** has no Test phase. It appears as `TrainingComplete = false`, and by default both its chambers are removed.

Where to look: the report's *Progressive ratio: training* table (one row per group, with its training end) and the training-flag notes beneath it.

5.2 Light QC: did the paired fly earn its light?

The firmware switches the light on from its own live reading of the paired sucrose well. pyflic counts licks afterwards, from a cleaned-up signal. Usually the two agree. When a sucrose well's resting signal slowly creeps up, however, the firmware sees a well that is constantly touched and fires the light on its own. pyflic sees a flat signal and no licks. The light then describes the sensor, not the fly, and every number built on it is meaningless. The light QC catches this, per group, over the whole recording.

Check	What it means	Verdict
Self-triggered light	5 (<code>pr_lick_free_run</code>) or more Test light events in a row with no sucrose lick since the previous one	fails the group
Implausible training	training "completed", with light events, but without a single sucrose lick	fails the group
No increasing trend	licks per light event don't rise across the Test phase, as a working progressive ratio should	warning
Resting level rise	the sucrose well's resting signal rose during the recording	warning
Resting level elevated	the sucrose well rests well above the DFM's other sucrose wells	warning

Some behaviour here is normal and is not flagged:

- **An occasional lick-free light event.** The firmware responds to brief touches below pyflic's feeding threshold. That is why only a *run* fails a group.
- **Fewer training licks than training light events**, for the same reason. Only *zero* licks is implausible.
- **A group with no Test light events at all** simply stopped before its first Test requirement. That is a breaking point of 0, not a failure.

A failed group is removed by default. `pr_light_qc.csv` still lists it, and the QC figures keep it on screen with **EXCLUDED** and the reason in its panel title, so you can check the decision. Warnings never remove a group; read them as reasons to look at the QC figures.

If you would rather judge a failed group yourself, `LickFreeRunStartMin` in `pr_light_qc.csv` is the minute after training end at which its failing run began. Two ways to use it:

- set `exclude_failed_pr_groups: false` and cap the analysis with `pr_test_window_min`;
- exclude the chambers by hand.

Whichever you choose, apply the same rule to every group, whatever its treatment.

5.3 Exclusions

Chambers leave the analysis for three kinds of reason:

- **By hand**, in `remove_chambers.csv`.
- **A chamber-level cutoff**: too few licks, an implausible duration or an implausible event count. Only that chamber is removed.
- **A group-level failure**: training never completed, or the light QC failed. Both chambers are removed.

Every automatic removal and its reason is written to `removed_chambers.csv` and listed in `summary.txt`. The Project gathers them all into `<project>_Excluded.csv`.

A group needs **both** of its chambers for anything that compares paired with yoked. If one chamber is removed, its partner still appears in the per-chamber tables, but the whole group drops out of the difference table, the difference curve and the breaking point.

5.4 The paired – yoked difference

For each group and each phase (Training, Test), pyflic subtracts the yoked fly's value from the paired fly's, for each metric:

Column	Paired – yoked ...
<code>dLicksA, dLicksB</code>	sucrose licks, yeast licks
<code>dEventsA, dEventsB</code>	sucrose feeding events, yeast feeding events
<code>dPI, dEventPI</code>	preference index (by licks, by events)
<code>dMedDurationA, dMedDurationB</code>	median feeding-event duration
<code>dPersistA</code>	sucrose persistence (section 5.7), Test rows only

Positive means the paired fly did more. **Zero is the null hypothesis:** the contingency made no difference beyond the light itself. The file is `paired_yoked_diff.csv`, one row per group per phase. The reports use the **Test** phase, where the requirement is rising.

5.5 The cumulative difference curve

This is the difference as it builds up over time: for each group, paired minus yoked cumulative sucrose licks, minute by minute from training end. Per treatment, pyflic draws the mean $\pm$ SEM across groups, over the span of time that **every** group covers, so the curve never jumps when one group's recording ends. The individual groups are drawn faintly behind it.

5.6 The breaking point

How far did the paired fly keep working?

pyflic reads the paired fly's Test light events in order from training end. The fly is taken to have **stopped at its first pause longer than `pr_break_gap_min`** (120 minutes by default). Pauses are counted from training end to the first event, between events, and from the last event to the end of the Test window.

The **breaking point** is the number of light events the fly completed before that pause: the last ratio it met. Some details:

- **Only lick-backed events count.** A lick-free light event neither counts nor ends a pause, because a light with no lick is no evidence that the fly was still working.
- **A pause of exactly the gap does not end the count.**
- **BreakMin** is the minute, after training end, of the last event counted.
- **Censored (n+).** If no long pause came before the Test window ended, the fly was still working when the recording stopped. Its count is a **lower bound**, not a measurement. Tables write it as **12+**, the dot plot

draws it as an open symbol, and the still-responding curve marks it with a tick.

- **The yoked fly has no breaking point.** Its light is its partner's, so there is nothing for it to have worked for.
- **The unit is light events, not licks.** The firmware's lick schedule isn't recorded in the data.

LargestRequirement, the most licks credited to one counted event, is given as a description only.

The file is **pr_breaking_point.csv**, one row per group, with **BreakingPoint**, **BreakMin**, **Censored**, **TestMinutes**, **LargestRequirement** and the group's light QC result. **pr_light_events.csv** lists every Test light event, with a **Counted** column showing which ones make up the breaking point. Use it to check any group by hand.

5.7 Sucrose persistence

The same rule applied to feeding, for **both** flies: the minutes from training end to the fly's last sucrose feeding event before a pause longer than **pr_break_gap_min**. It is censored the same way when the fly was still feeding at the end of the window.

Because the yoked fly has it too, it has a paired – yoked difference, **dPersistA**. **dPersistCensored** marks differences where either fly was censored. Those differences are kept and flagged, not dropped, because in a recording that ends during a feeding peak, dropping them would remove most groups.

Persistence is in **PersistA** / **PersistACensored** on every per-chamber summary row, and in **dPersistA** / **dPersistCensored** in the difference table.

6. Reading the figures

Results figures

Cumulative difference curve (**pr_cumulative_diff.png**). The line is each treatment's mean paired – yoked cumulative sucrose licks since training end, with the shaded band its SEM, over the time every group covers. The faint lines are individual groups, and the dashed line is zero.

- A line climbing above zero means the paired flies worked for the light more than their yoked partners fed.
- A line that flattens shows when the difference stopped growing.
- A flat line near zero means no operant effect.
- Check the faint lines: one extreme group can carry a mean.

Paired – yoked dot plots (reports). One point per chamber group, for Test-phase **dLicksA**, **dPI** and, when present, **dPersistA**, with the treatment mean $\pm$ SEM and a line at zero. Points mostly above zero support a paired > yoked effect. Compare how far the treatments sit from zero, not only from each other.

Breaking point by treatment (reports). One point per group, with mean $\pm$ SEM. **Open symbols are censored**: the true value is at least that high. If one treatment has many open symbols, its mean understates it.

Still-responding curve (**pr_still_responding.png**). For each treatment, the fraction of paired flies that reached each ratio: 1.0 at ratio 0, stepping down as flies stop. Ticks mark censored flies, which were still responding when their window ended. The legend gives each treatment's *n*.

- A curve lying to the right of another means that treatment's flies kept working to higher ratios.
- This is the figure that treats censoring correctly. When many flies are censored, trust it over the dot plot.

Breaking-point plots ([breaking_point_dfm<id>.png](#), from the Hub's **Breaking-point plots** button and in the experiment report). One panel per chamber, both roles, showing the licks between successive light onsets over time since training end.

- Blue points are the events the breaking point counts.
- Grey points came after the break, or after the Test window.
- Hollow red rings are lick-free events, never counted.
- The dashed line marks the break. A censored group has no dashed line, and its strip reads **BP n+**.
- Groups removed by the light QC are drawn for reference, with "— excluded" in the strip and no break marked.

Quality-control figures

Licks per light event ([pr_light_events_dfm<id>.png](#)). For each group, the sucrose licks credited to each Test light event, in order.

- **A working progressive ratio climbs.**
- Hollow red rings are lick-free events; a run of them along the bottom is self-triggered light.
- The dashed line is the group's own trend. The grey line is the requirement pyflic estimates across the experiment, for reference.

Sucrose Well resting level ([pr_resting_level_dfm<id>.png](#)). For each group, the paired sucrose well's resting signal over the whole recording, against the median of the DFM's other sucrose wells. The rug along the bottom shows light onsets, and the dashed line is training end.

- A well that creeps up while the rug gets denser is a sensor problem, not a motivated fly.
- This figure is usually the clearest explanation of a self-triggered failure.

Training-aligned traces ([pr_cumulative_licks_dfm<id>.png](#)). Paired and yoked cumulative sucrose licks since each group's training end. Points mark minutes with the light on; black rings mark lick-free light events on the paired trace. Use it to see whether the light kept firing while the paired trace went flat. That pattern is self-triggering.

Every QC figure's panel title gives the group's treatment and its light QC result: *light QC ok*, *warning: ...*, or *EXCLUDED: ...*.

7. Statistics and inference

The unit of analysis

One observation per chamber group. Paired and yoked flies are never entered as separate observations in the main tests. The yoked fly is the paired fly's control, and pooling them would average an effect with its own control.

The tests

Question	Test	Where
Is the paired fly different from its yoked partner?	Each treatment's Test-phase differences against zero: the paired t-test (a one-sample t-test on the differences), with the Wilcoxon signed-rank test beside it; in a Project, also a mixed model's intercept	both reports; <project>_Stats.txt
Do treatments differ in the paired – yoked difference?	Welch's t-test for two treatments, Tukey HSD for three or more; in a Project, also a mixed model	both reports; <project>_Stats.txt
Do treatments differ in breaking point?	The same tests on BreakingPoint , plus a pairwise log-rank test on the ratio reached	both reports; <project>_Stats.txt
What fraction of flies reached each ratio?	Kaplan-Meier estimate (the still-responding curve)	both reports; pr_still_responding.png

The per-chamber pooled tests from the standard two-well analysis are still reported, as a secondary section.

In a Project with two or more members, the **mixed model** treats each experiment, and each DFM within it, as a source of variation of its own (DFM nested within Experiment). It is reported beside the pooled p-value.

When they disagree, prefer the mixed model: it doesn't treat groups on the same device, or in the same recording, as independent.

What a result lets you say

- **A difference above zero in the paired flies** (for example **dLicksA** in the Test phase) supports the light acting as a reinforcer. The paired fly fed more when feeding produced the light than a fly given the same light regardless. The yoked design controls for the light's direct effects on feeding, but only when both flies of a group were treated identically in every other way.
- **A higher breaking point** means the paired flies kept working to higher requirements, a measure of how much effort the light was worth. There is no yoked comparison for it. Compare treatments with each other, and keep the dot plot and the still-responding curve side by side.
- **A difference between treatments in the paired – yoked difference** means the treatment changed how reinforcing the light was, not merely how much the flies fed.

Cautions before you conclude

1. **Censored breaking points.** The t-test, Tukey and the mixed model enter a censored count as if it were the true value, which **biases that treatment's mean downward**. The log-rank test and the still-responding curve treat censoring correctly. Count the censored groups per treatment (open symbols, **n+**). When censoring is common or uneven between treatments, base the conclusion on the log-rank test and the curve, and consider a longer recording.
2. **The gap setting.** Check the sensitivity table (60 / 120 / 240 minutes). A conclusion that holds at only one gap is weak and should be reported as such.

3. **Unequal Test windows.** Groups that trained late had less Test time. If `TestMinutes` varies widely, set `pr_test_window_min` so every group is judged over the same span.
4. **Independence in a single experiment.** In one experiment's report, the three groups on a DFM are treated as independent although they share a device. The mixed model that accounts for this runs only across two or more members, so replicate across recordings and pool them in a Project.
5. **Multiple comparisons.** The reports test several metrics (`dLicksA`, `dLicksB`, `dEventsA`, `dPI`, `dMedDurationA`, `dPersistA`, and `dEventsB` in a Project) without correction. Tukey HSD corrects within one metric; the log-rank tests are **not** corrected between treatment pairs. Decide your primary measure before you look, and treat the others as supporting.
6. **Exclusions can bias a comparison.** If the light QC or training removes more groups from one treatment than another, the remaining groups may not be comparable. `<project>_Excluded.csv` and the report's exclusion table give the counts. Report them per treatment.
7. **Lick detection matters.** Whether a light event counts depends on pyflc detecting at least one lick. Detection parameters that miss brief licks turn real responses into lick-free events. That lowers breaking points and can trigger the self-triggered check. Keep detection parameters identical across the Project, and look at the QC figures when many lick-free events appear.
8. **Sucrose persistence differences can be differences of lower bounds.** Rows with `dPersistCensored = true` are kept. If many are censored, interpret `dPersistA` cautiously.

What to report

- The number of chamber groups per treatment, before and after exclusion, with the reasons.
- `pr_break_gap_min`, `pr_test_window_min` (if used), and the detection parameters.
- The number of censored groups per treatment.
- For the breaking point: the still-responding curve and the log-rank test, with the mean comparison as secondary.
- For the paired – yoked difference: the test against zero within each treatment, and the between-treatment comparison.
- Whether the conclusion survives the breaking-point sensitivity table.
- For pooled data: the mixed-model p-values.

8. Quick troubleshooting

You see	Likely cause	What to do
Every group removed, with "training never completed"	a v2 file or no light data: no training flags	check the DFM files are version 3 with <code>OptoColl1</code>
"no training flag on any well of group <i>g</i> "	same as above, or the run never entered training	as above
A group failed for self-triggered light	the sucrose well's resting level drifted, so the light fired on its own	look at <i>Sucrose Well resting level</i> ; the group is correctly excluded
A group failed for implausible training	training was completed by the sensor, not the fly	check the well and the fly; the group is correctly excluded

You see	Likely cause	What to do
Almost every breaking point is censored (n+)	the Test window is short compared with <code>pr_break_gap_min</code>	record longer, or reconsider the gap; don't shorten the gap to get a result
A group is missing from the difference table	one of its two chambers was removed	see <code>removed_chambers.csv</code>
The Project Report says a member has no breaking point table	that member was analysed with an older pyflic	re-run its basic analysis
Loading fails on <code>paired_chambers</code>	missing, two chambers from one group, or a group left out	name exactly one chamber from each of 1–2, 3–4, 5–6
Loading fails on treatments	the two chambers of a group have different treatments	give both chambers the same treatment